Zużycie energii – monitorowanie

- MSR – przykład
  - Odczyt rejestru o znanym adresie
  - rdmsr (msr-tools)

```
int read_msr(int cpu, unsigned int address, uint64_t *value)
{
   int err = 0;
   char msr_path[32];
   FILE *fp;

   sprintf(msr_path, "/dev/cpu/%d/msr", cpu);
   err = ((fp = fopen(msr_path, "r")) == NULL);

   if (!err)
      err = (fseek(fp, address, SEEK_CUR) != 0);
   if (!err)
      err = (fread(value, sizeof(uint64_t), 1, fp) != 1);
   if (fp != NULL)
      fclose(fp);

   return err;
}
```

Zużycie energii – monitorowanie

- Skrypt - odczyt liczników – używa **rdmsr** (patrz poprzedni slajd)

```
#!/bin/bash
SAMPLING_RATE=1                    # seconds
MSR_PKG_ENERGY_STATUS="0x611"    # CPU energy counter
MSR_DRAM_ENERGY_STATUS="0x619"  # DRAM energy counter

# Energy Status Units (ESU, typ. 15.3uJ)
ESU=`echo "ibase=16; 1/2^$(rdmsr -X 0x606 -f 12:8)" | bc -l`

# CPU energy status counter incremants
ESPKG=`a=$(rdmsr -X $MSR_PKG_ENERGY_STATUS);\
       sleep $SAMPLING_RATE;\
       echo "ibase=16;$(rdmsr -X $MSR_PKG_ENERGY_STATUS)-$a" | bc`

# DRAM energy status counter incremants
ESDRAM=`a=$(rdmsr -X $MSR_DRAM_ENERGY_STATUS);\
        sleep $SAMPLING_RATE;\
        echo "ibase=16;\$(rdmsr -X $MSR_DRAM_ENERGY_STATUS)-$a" | bc`

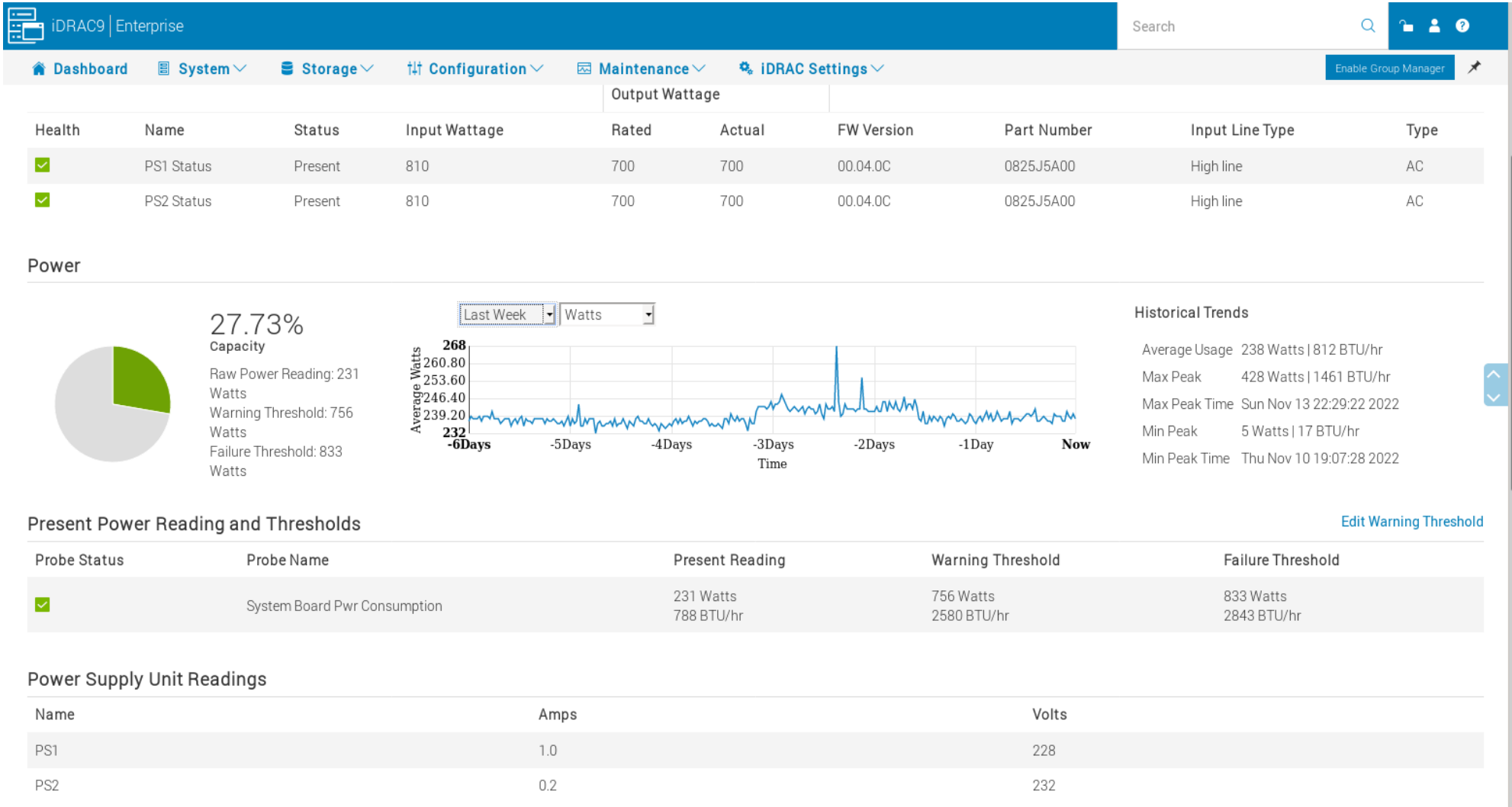
# Calculate power consumption [W]
CPUPOW=`echo "$ESPKG * $ESU" | bc -l`
DRAMPOW=`echo "$ESDRAM * $ESU" | bc -l`
echo CPU: $CPUPOW W
echo DRAM: $DRAMPOW W
```

Zużycie energii – monitorowanie

- IPMI (*Inteligent Platform Management Interface*)
  - serwer musi być wyposażony w BMC (Dell: idrac, HP: ILO…)

Zużycie energii – monitorowanie

- IPMI (*Inteligent Platform Management Interface*)
  - serwer musi być wyposażony w BMC (Dell: idrac, HP: ILO…)
  - przykład – interfejs www idrac 9



Zużycie energii – monitorowanie

- IPMI (*Inteligent Platform Management Interface*)
  - serwer musi być wyposażony w BMC (Dell: idrac, HP: ILO...)
  - możliwy odczyt parametrów z zasilacza
    - pobór mocy
    - napięcia
    - temperatura
  - podobne parametry z pozostałych komponentów (CPU, RAM, dyski, wentylatory ...)
- Przykład (Dell/ssh):

```
linux$ ssh root@192.168.1.1 racadm get System.Power.Realtime.Power
root@192.168.1.1's password:
244 W | 833 Btu/hr
```

- Również: ipmitool

ACPI

- ACPI (*Advanced Configuration and Power Interface*)
- Funkcje
  - zarządzanie konfiguracja urządzeń – PnP (P*lug and Play*)
  - zarządzanie poborem mocy
    - koncepcja hierarchii stanów energetycznych
    - stany globalne G* - cały komputer
      - stany S* - poziom „uśpienia” komputera
        - stany C* - poziom aktywności procesora (*idle states*)
          - stany P* - sterowanie częstotliwością i napięciem (*performance states*)
    - wykorzystane techniki
      - wyłączanie (części) urządzeń
      - sterowanie częstotliwością i napięciem

DVFS

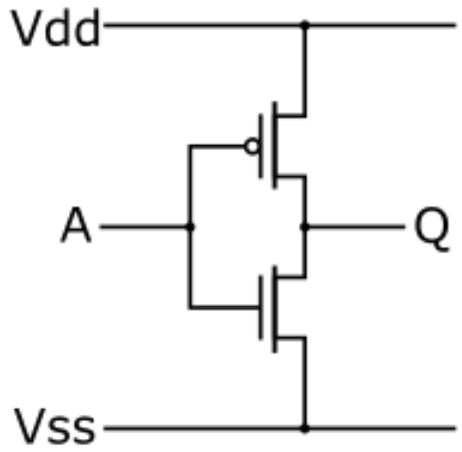
- DVFS (*Dynamic Voltage and Frequency Scaling*)
- Dobór częstotliwości i napięcia do poziomu wykorzystania procesora
- Pobór mocy przez procesor – model energetyczny bramki CMOS:

$$P = P_{leak} + P_{short} + P_{dyn}$$

$$P_{dyn} = \alpha C U^2 f \qquad P_{leak} \sim U \qquad P_{short} \sim f$$

P	– moc
P_{leak}	– moc tracona w wyniku upływu w stanie ustalonym
P_{short}	– moc tracona w wyniku upływu przez tranzystory w chwili przełączania
P_{dyn}	– moc potrzebna do przełączenia tranzystorów
C	– pojemność
U	– napięcie
f	– częstotliwość
α	– intensywność przełączeń na takt

- W działającym procesorze dominuje P_{dyn}
- Obniżenie częstotliwości pozwala na obniżenie napięcia



https://pl.wikipedia.org/wiki/Plik:CMOS_Inverter.svg

ACPI – stany G*/S*

- G0 – *working* – działa normalnie
  - S0 – *working*
- G1 – *sleeping* – uśpiony
  - S0ix – *modern standby*
  - S1 – *power on suspend* – wstrzymane wykonanie programu, CPU i RAM zasilane
  - S2 – wyłączone zasilanie CPU
  - S3 – *suspend to RAM*
  - S4 – *suspend to disk* – hibernacja
  - ...
- G2 – *soft off* – wyłączony programowo (soft off)
  - S5 – *soft off*
- G3 – *hard off* – wyłączony fizycznie

ACPI – stany procesora C*/P*

- W stanie G0/S0 procesor może być w następujących stanach:
  - C0 - *working*
    - P0 – maksymalna częstotliwość i pobór mocy
    - P1 – zmniejszone częstotliwość i pobór mocy
    - P2, P3 … Pn
    - wydajność i pobór mocy maleje monotonicznie
      - liczba stanów P* zależy od konstrukcji procesora
  - C1 – *halt* – może „błyskawicznie” powrócić do C0
  - C2 – *stop clock*
  - C3 – *sleep – not cache coherent*
  - Liczba stanów zależy od konstrukcji procesora
  - Kolejne stany wymagają dłuższego czasu na „wybudzenie”

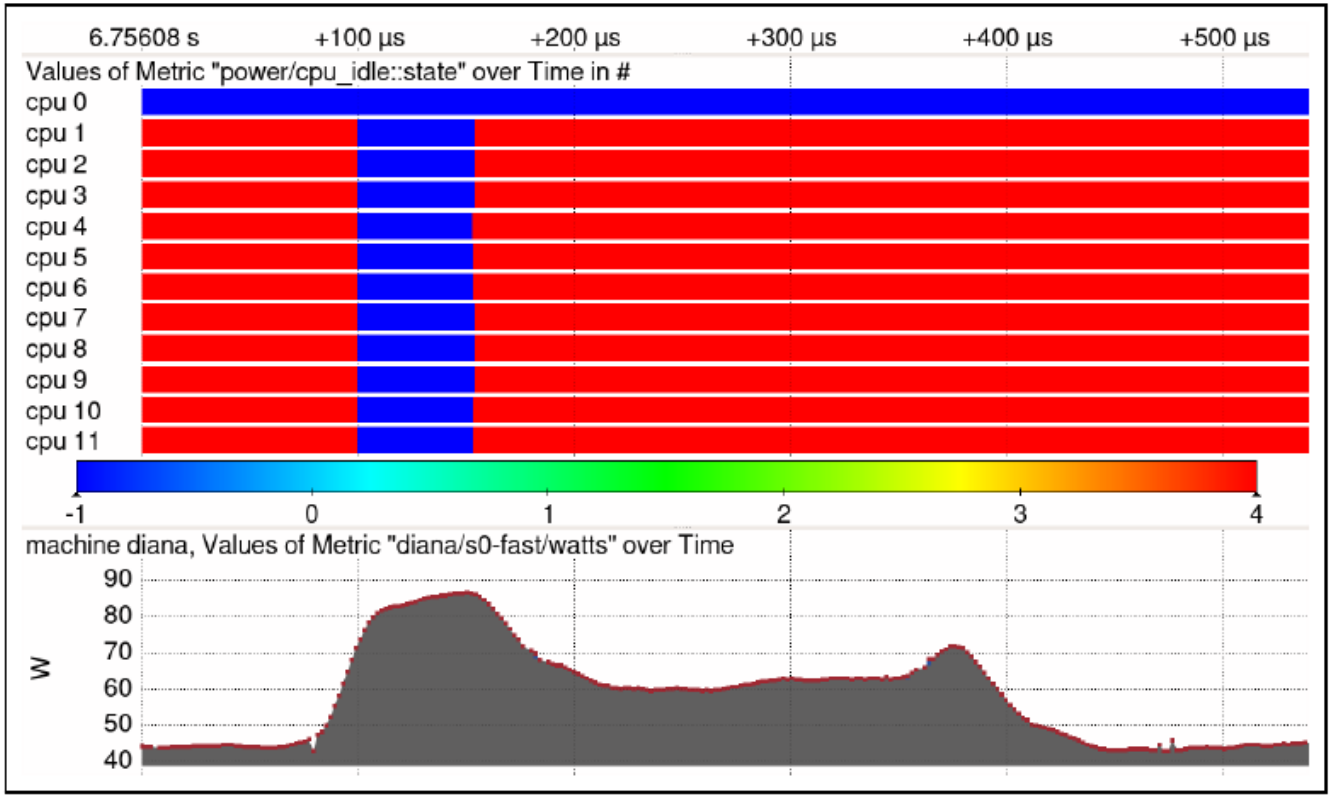
Stany procesora – przykład

- Pobór mocy przez procesor Pentium M 1600MHz

Stan C	Stan P	Częstotliwość [MHz]	Napięcie [V]	Moc [W]
C0	P0 P1 P2 P3	1600 1300 1100 600	1,484 1,388 1,18 0,956	24,5 22 12 6
C1, C2	P0-P2 P3	0 0	1,484 0,956	7,3 1,8
C3	P0-P2 P3	0 0	1,484 0,956	5,1 1,1
C4	-	0	0,748	0,55

Stany procesora – przykład

- Czas przełączenia

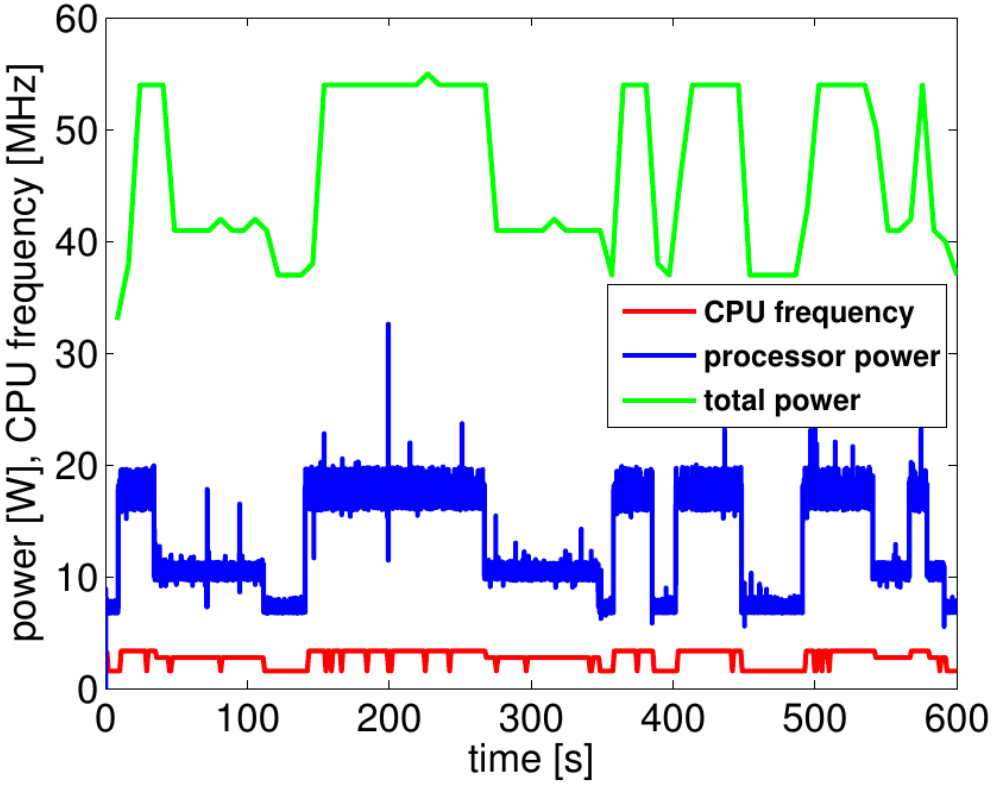


(b) Aligned wake-up of all CPUs. In the upper metric display, the C-state of all CPUs of the first socket is visualized (blue: active, red: C6). After all CPUs switch to idle, the power consumption remains high for ≈230 µs

zródło: Thomas Ilsche, Robert Schöne, Philipp Joram, Mario Bielert, Andreas Gocht, System Monitoring with lo2s: Power and Runtime Impact of C-State Transitions, IPDPS Workshops 2018: 712-715

Stany procesora – sterowanie w Linuksie

- Wybór stanu C: *idle_governor*
  - *menu – estymacja czasu bezczynności*
  - *timer events oriented - prostsza wersja*
  - *ladder – kolejne stany*
  - *halt polling – łagodniejszy dla vCPU*
- Wybór stanu P: *frequency_governor*
  - sterownik intel_pstate
    - performance
    - powersave
  - sterownik acpi_cpufreq
    - performance – maks. częstotliwość
    - powersave – min. częstotliwość
    - ondemand
    - conservative
    - userspace – np. **cpufreq-set -f 2GHz**



- Ustawiany dla każdego procesora (rdzenia) indywidualnie

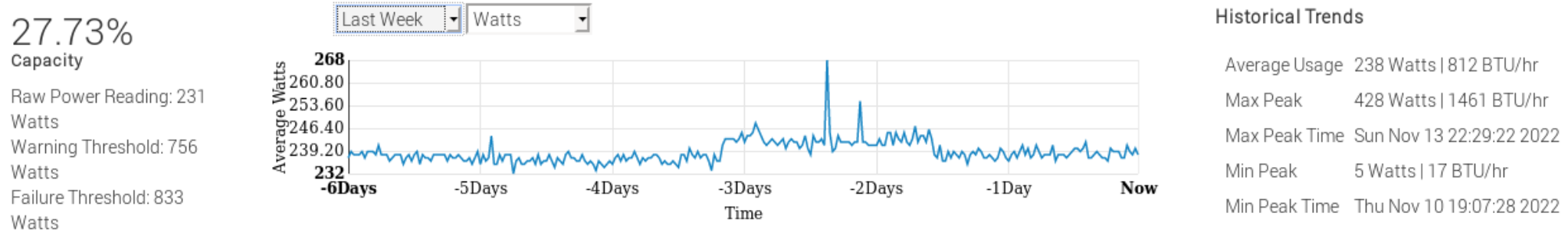
```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_governors

echo performance > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

również: **`cpufreq-info`**

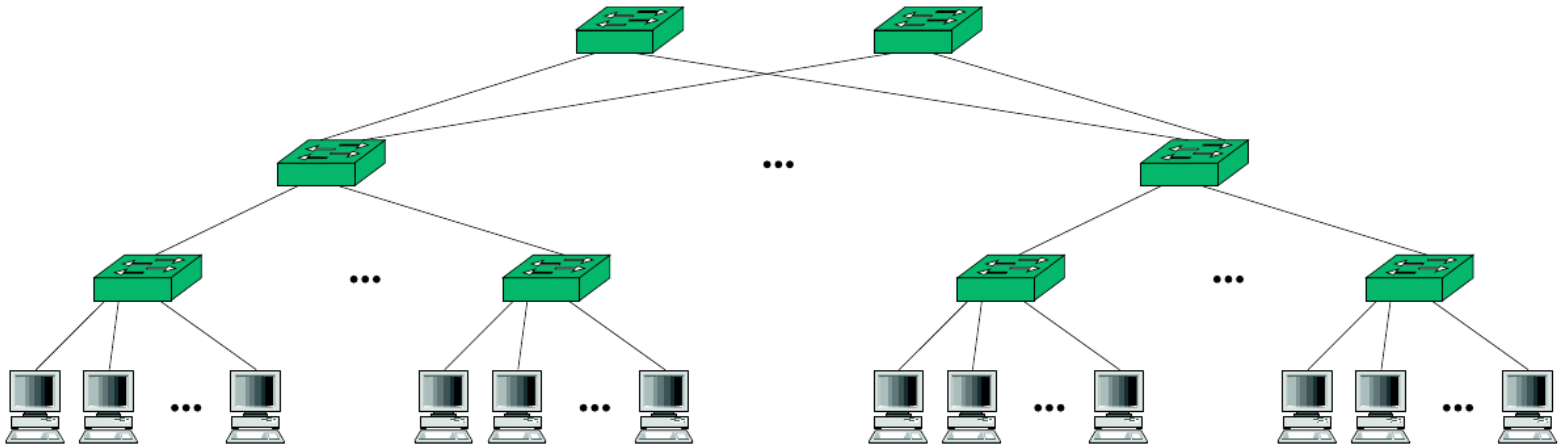
Algorytmy lokalne a polityki globalne

- Algorytmy lokalne, np. sterowanie CPU
  - sterują pojedynczym urządzeniem (cpu, interfejs sieciowy)
  - krótka skala czasowa
  - brak (zazwyczaj) wiedzy o naturze zadań
- Polityki globalne
  - wiedza o naturze zadań – np. prognoza obciążenia
  - długa skala czasowa
  - sterują całym systemem komputerowym, klastrem
    - wiedza o budowie klastra
    - koordynacja działań pozwala uzyskać lepsze wyniki



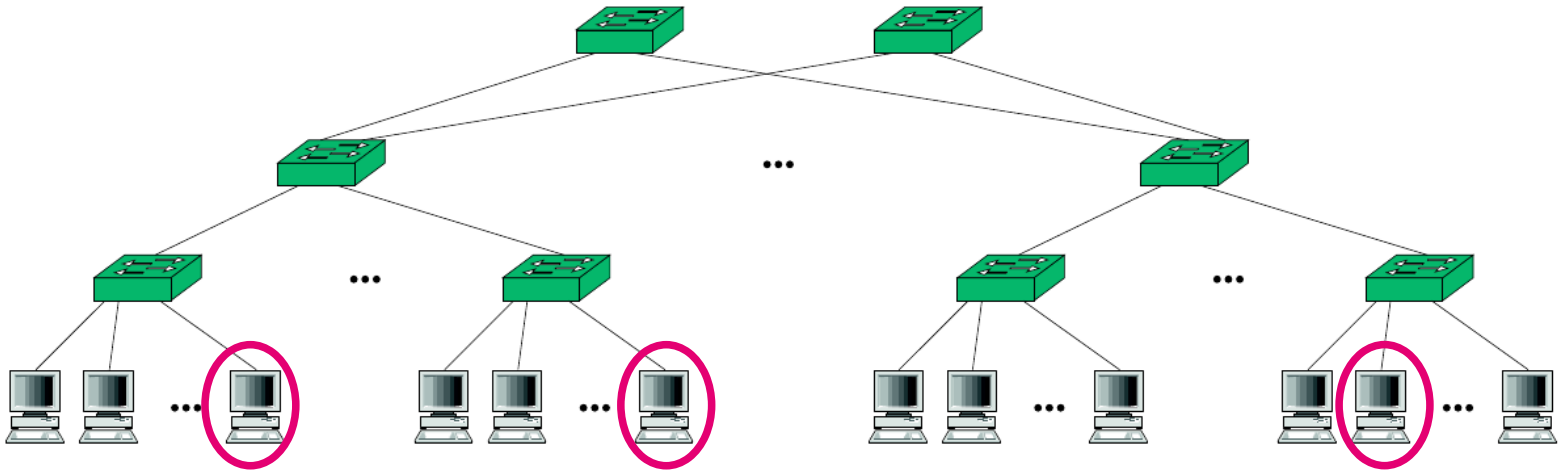
Algorytmy lokalne a polityki globalne

- koordynacja zadań pozwala uzyskać lepsze wyniki
  - przy rozrzuconych zadaniach algorytm lokalny obniży zegar serwerów



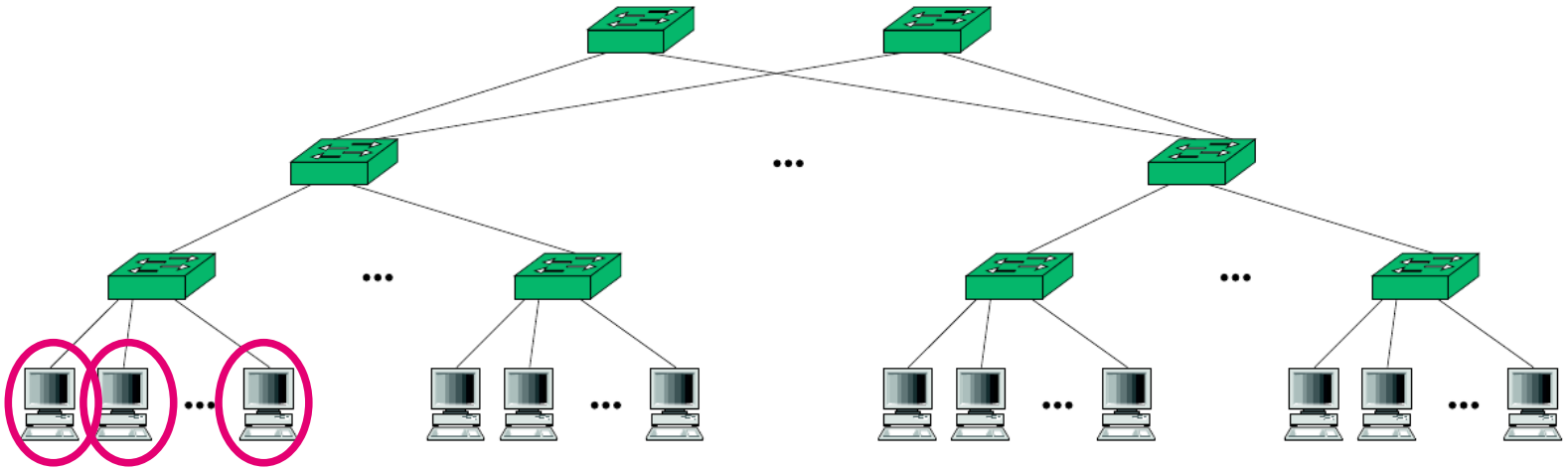
Algorytmy lokalne a polityki globalne

- koordynacja zadań pozwala uzyskać lepsze wyniki
  - np. wyłączanie serwerów/konsolidacja obciążenia



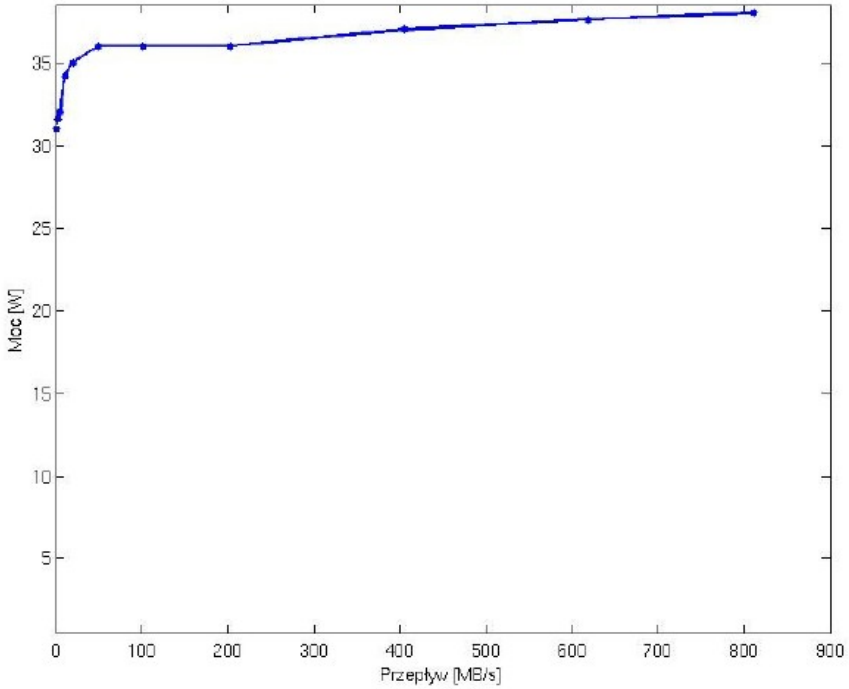
Algorytmy lokalne a polityki globalne

- koordynacja zadań pozwala uzyskać lepsze wyniki
  - np. wspólna alokacja cpu i pasma



Dlaczego konsolidacja działa?

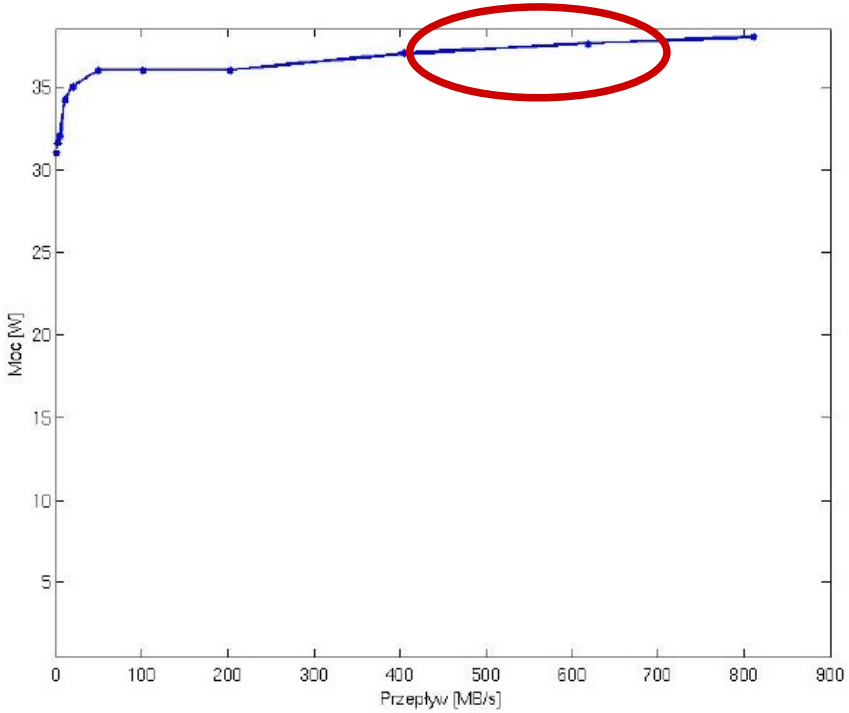
- Power proportionality
- Pobór mocy nie jest proporcjonalny do obciążenia
  - próg włączenia
  - nieliniowy przebieg
- Algorytmy lokalne dążą do zwiększenia proporcjonalność



Ruter programowy PC

Dlaczego konsolidacja działa?

- Power proportionality
- Pobór mocy nie jest proporcjonalny do obciążenia
  - próg włączenia
  - nieliniowy przebieg
- Algorytmy lokalne dążą do zwiększenia proporcjonalność



Ruter programowy PC

- Konsolidacja pozwala działać w zakresie największej efektywności

Dlaczego warto stosować polityki globalne?

- Wykorzystanie sprzętu jest na ogół niskie (mniejsze niż 50%)
- Zmienność obciążenia w długiej skali
  - dobowa, tygodniowa
  - wynika z natury zadań, trybu pracy użytkowników
  - możliwe prognozowanie
- Redundacja
  - zwiększenie niezawodności
  - zapewnienie wolnej mocy obliczeniowej i pasma (tzw. *headroom*)
    - np. Ethernet używa CSMA/CD

Wykorzystanie serwerów

typ	2000-2010	2020
internal	10%	15%
service provider	20%	25%
hyperscale	45%	50%

A. Shehabi, S. Smith, D. Sartor, R. Brown, M. Herrlin, United States Data Center Energy Usage Report, Lawrence Berkeley National Laboratory, 2016

Algorytmy alokacji zadań

- Predykcyjne
  - Wykorzystują wiedzę o wykorzystaniu zasobów w przeszłości
  - Działają w dłuższej skali czasowej
- Reaktywne
  - Reagują na zmiany obciążenia
  - Dążą do utrzymania punktu pracy

Przykład zadania alokacji

- N zadań
- S_f procesorów
- K_s stanów energetycznych
- Przydział statyczny, tzn. zadania działają stale
- Cel: rozdzielenie zadań i minimalizacja mocy
- Charakterystyka zadań:
  - W_n – zapotrzebowanie na moc obliceniową [MIPS]
  - M_n – zapotrzebowanie na pamięc [MB]
- Chararakterystyka procesora
  - Θ_s^k - moc obliczeniowa [MIPS]
  - Ψ_s - pamięć [MB]
  - możliwy *overbooking*

Przykład zadania alokacji

- Funkcja celu – minimalizacja poboru mocy: S_f procesorów, K_s stanów energetycznych

$$\min_{\mathbf{x_s},\vartheta_s}\left[\sum_{s=1}^{S_f}\sum_{k=1}^{K_s} P_s^k x_s^k\right]$$

- ograniczenia

$$\forall_{s=1,\ldots,S_f} \quad \sum_{k=1}^{K_s} x_s^k = 1$$ – jeden stan energetyczny

$$\forall_{s=1,\ldots,S_f} \quad \sum_{n=1}^{N} W_n \vartheta_{sn} \le \sum_{k=1}^{K_s} \Theta_s^k \bar{W}^k x_s^k$$ – zdolność obliczeniowa procesora

$$\forall_{s=1,\ldots,S_f} \quad \sum_{n=1}^{N} M_n \vartheta_{sn} \le \sum_{k=1}^{K_s} \bar{M} \Psi_s$$ – dostępna pamięć

$$\forall_{n=1,\ldots,N} \quad \sum_{s=1}^{S_f} \vartheta_{sn} = 1$$ – wszystkie zadania przypisane

$\bar{W}^k$, $\bar{M}$ - współczynniki overbookingu

Przykład zadania alokacji

- Problemy
  - Zmienne binarne
  - Rozmiar zadania:
    - Liczba zmiennych: $S_f\ (\ K_s + N)$
    - Liczba ograniczeń: $3S_f + N$

    - np. mały klaster 144 rdzenie, 5 stanów energetycznych, 200 zadań:
      - 144(5+200)=29520, 3*144+200=632

    - klaster z przykładu miał 32 cpu 32 rdzeniowe czyli 1024 rdzenie, przy 2000 zadań
      - 1024*(5+2000)≈2mln, 3*1024+2000=5072

    - eksplozja wymiarowości

Przykład zadania alokacji

- Problemy, modyfikacje
  - Przydział zadań przy przetwarzaniu wsadowym
    - zazwyczaj potrzebne nieliniowe ograniczenie na czas obliczeń
  - Energooszczędne sterowanie siecią
    - Ruting: dodatkowe zmienne
    - Trudne zadanie – wyznaczanie najkrótszych ścieżek
    - Przykład – jedno z dodatkowych ograniczeń – warunek ciągłości przepływu dla węzłów wewnątrz sieci:

$$\forall_{\substack{r\in\Omega,\\ j=1,\ldots,J,\\ d=1,\ldots,D_j}} \quad \sum_{c=1}^{C} y_{rc}\left(\sum_{p=1}^{P} u_{cp}\sum_{l=1}^{L} a_{lp}v_{ljd} - \sum_{p=1}^{P} u_{cp}\sum_{l=1}^{L} b_{lp}v_{ljd}\right) = 0$$

    - Bilans dla każdego węzła r (C kart , P portów), zadania j , przepływu d
    - Nowe zmienne, wymiar „najciekawszej” – ljd

Przykład zadania alokacji

- Problemy, modyfikacje
  - Przydział zadań przy przetwarzaniu wsadowym
    - zazwyczaj potrzebne nieliniowe ograniczenie na czas obliczeń
  - Energooszczędne sterowanie siecią
    - Ruting: dodatkowe zmienne
    - Trudne zadanie – wyznaczanie najkrótszych ścieżek
    - Przykład – jedno z dodatkowych ograniczeń – warunek ciągłości przepływu dla węzłów wewnątrz sieci:

$$\forall_{\substack{r\in\Omega,\\ j=1,\ldots,J,\\ d=1,\ldots,D_j}} \quad \sum_{c=1}^{C} y_{rc}\left(\sum_{p=1}^{P} u_{cp}\sum_{l=1}^{L} a_{lp} v_{ljd} - \sum_{p=1}^{P} u_{cp}\sum_{l=1}^{L} b_{lp} v_{ljd}\right) = 0$$

    - Bilans dla każdego węzła r (C kart , P portów), zadania j , przepływu d
    - Nowe zmienne wymiar „najciekawszej” – ljd

Algorytmy reaktywne

- BFD – Best Fit Decreasing
  - VM posortowane od wymagających największych zasobów
  - Przypisywane do serwerów mających wystarczające zasoby
- PCA-BFD – Power and Computation Capacity BFD
  - Uwzględnia wydajność energetyczną serwera
- LPBP – Low Perturbation Bin Packing
  - Migruje VM z serwera o największym poborze mocy na pobierający najmniej
- Tzw. trywialne – np. Firtst Fit, Worst Fit

- Miary efektywności energetycznej
  - PPW – Performance Per Watt, np. Hz/W, MIPS/W, Mbps/W
- Load Consolidation – pozwala wyłączyć/uśpić część zasobów

VMware ESXi – architektura

- Wirtualizator typu 1 (*bare metal*)
- Pełna virtualizacja z wsparciem sprzętowym
  - wymagania – (VT-X), lista kompatybilnego sprzętu
- VMKernel – lekkie (32MB) jądro, POSIX

